

High Availability (HA) Clusters for AI Infrastructure

Building resilient AI systems with no single point of failure

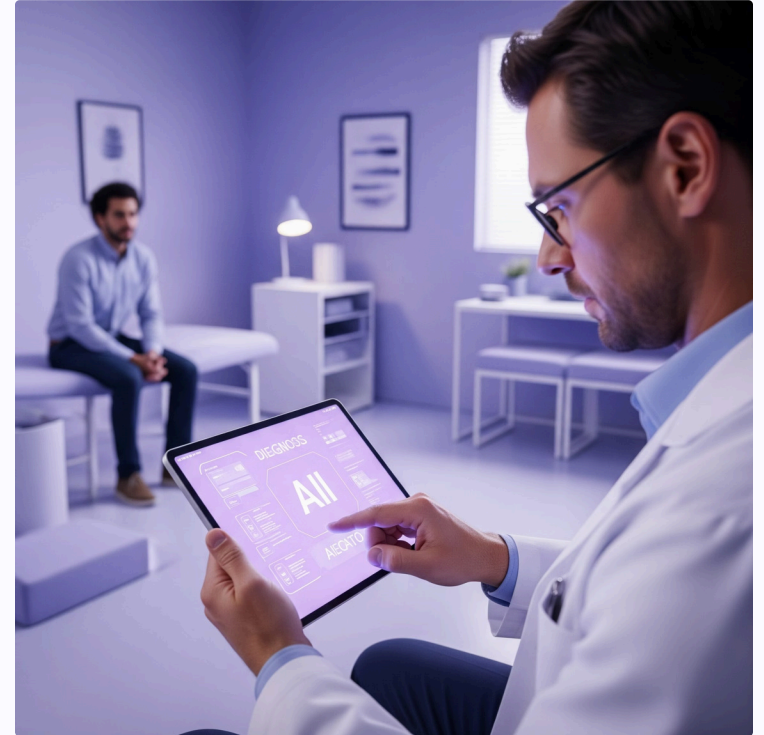


Why HA Matters in AI

AI APIs often power **real-time critical services** where reliability is non-negotiable:

- Fraud detection systems that protect financial transactions
- Healthcare diagnostics informing treatment decisions
- Self-driving vehicle operations requiring constant availability

Outages lead to lost revenue, damaged reputation, and potential regulatory violations.



Our capstone goal: $\geq 99.9\%$ uptime SLA

High Availability Principles



Redundancy

Replicate critical services and infrastructure components to eliminate single points of failure



Failover

Automatically detect failures and reroute traffic to healthy replicas with minimal disruption



Isolation

Prevent failures from cascading across systems with proper boundaries and circuit breakers



Self-healing

Automatically restart failed components or reschedule workloads without human intervention

HA Layers in AI Infrastructure

Compute

Multi-node GPU clusters with autoscaling capabilities to handle varying workloads

Storage

Data replication, multi-zone buckets, and consistent snapshot policies

Networking

Redundant load balancers, DNS failover mechanisms, and resilient connectivity

Control Plane

Multi-master Kubernetes clusters with distributed consensus protocols

Application

Canary deployments, retry mechanisms, and circuit breakers for graceful degradation

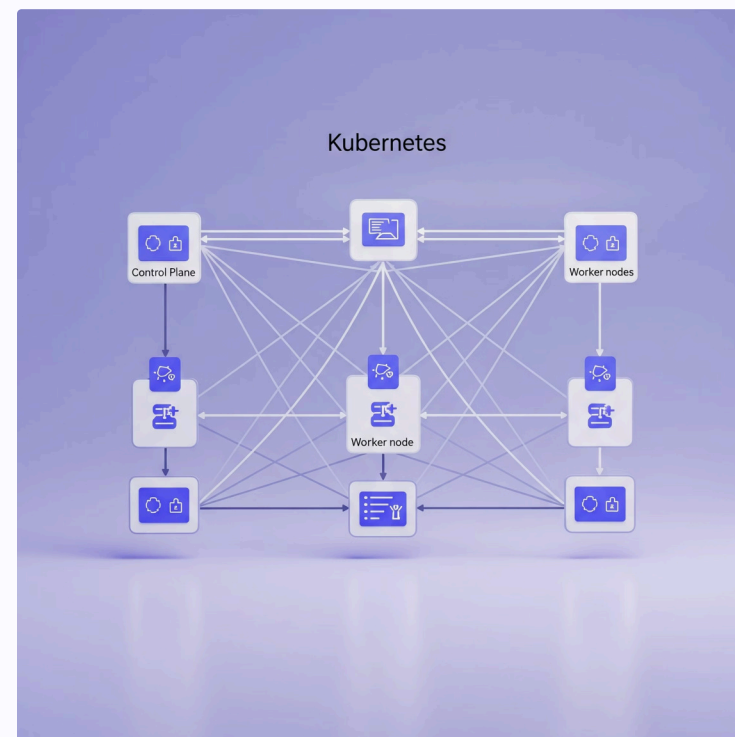
Kubernetes HA Setup

Control Plane Redundancy

- 3-5 control plane nodes distributed across availability zones
- Etcd cluster replicated for consistent distributed state

Worker Node Distribution

- Worker nodes spread across **at least 2 Availability Zones**
- PodDisruptionBudgets (PDB) to maintain quorum during updates
- Cluster Autoscaler coupled with strategic Node Pools



Pod Resilience Strategies

Key Pod Resilience Techniques

- **ReplicaSets/Deployments** with ≥ 3 replicas for redundancy
- **PodAntiAffinity** rules to spread replicas across nodes and AZs
- **PodDisruptionBudgets** to maintain quorum during maintenance
- **Readiness/Liveness probes** for automatic healing

Example: Anti-Affinity YAML

```
affinity:  
  podAntiAffinity:  
    requiredDuringSchedulingIgnoredDuringExecution:  
      - labelSelector:  
          matchLabels:  
            app: infer  
        topologyKey: "kubernetes.io/hostname"
```

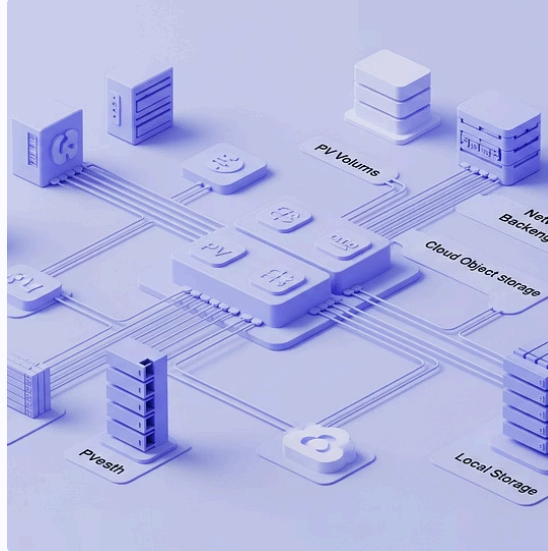
This ensures inference pods are scheduled on different physical nodes, preventing correlated failures.

Storage HA Strategies



Cloud-Native Storage

- AWS S3/GCS/Azure Blob with cross-zone replication
- Automated snapshot policies for point-in-time recovery
- Versioning for accidental deletion protection



Kubernetes Volumes

- Storage classes with built-in replication
- StatefulSets for ordered, persistent workloads
- Volume snapshots for backup/restore



Database Clustering

- PostgreSQL with Patroni for automated failover
- Distributed systems like Cassandra or MongoDB Atlas
- Read replicas for performance and availability

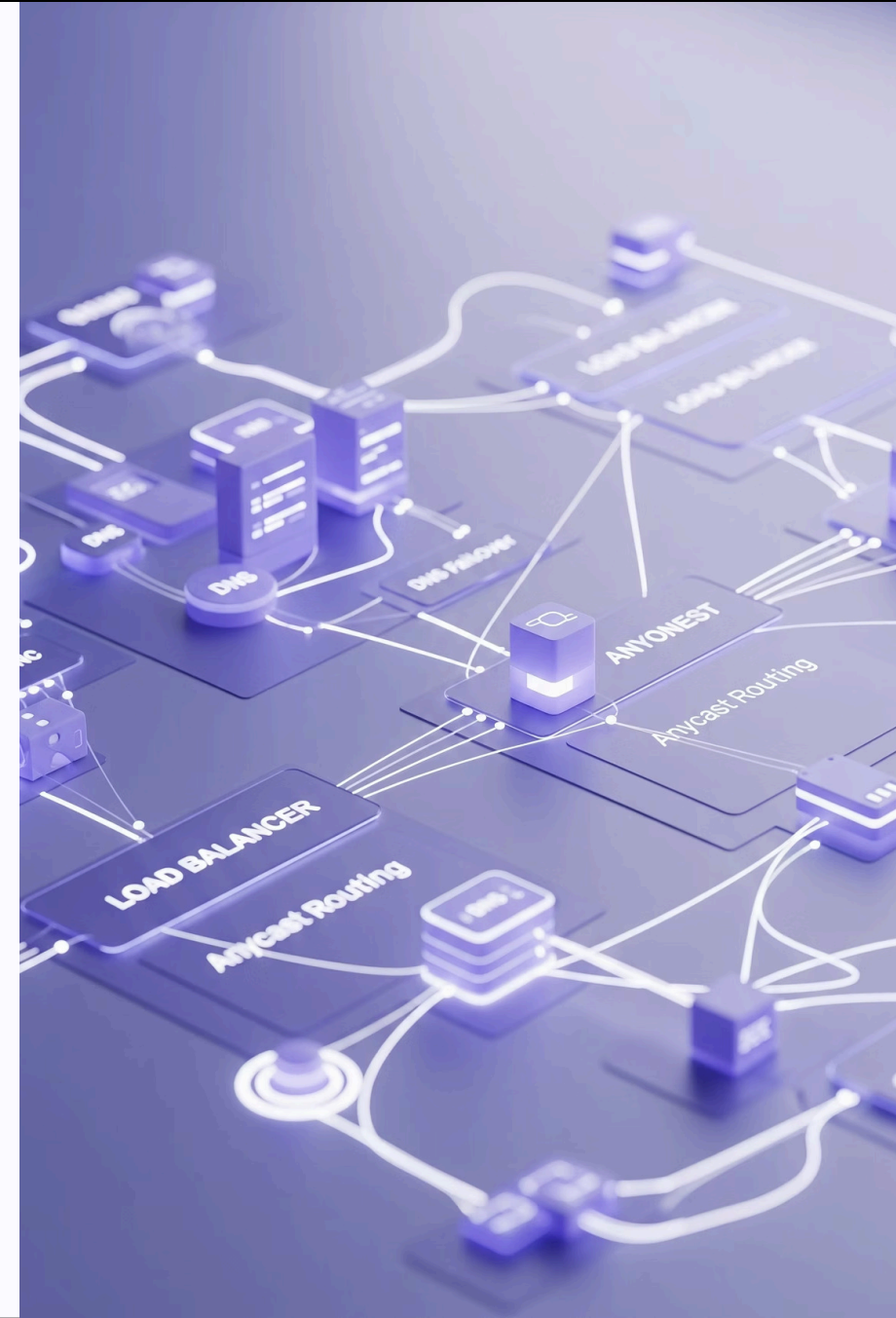
Networking HA

Load Balancing

- **Layer 4/7 Load Balancers** distributed across zones
- Health checks with configurable thresholds and timeouts
- Session persistence when needed for stateful workloads

Global Routing

- **DNS failover** (Route53, Cloud DNS, Traffic Manager)
- Global Anycast IPs for latency optimization and failover
- **Ingress controllers** with minimum 2 replicas



Multi-Zone & Multi-Region Design



Multi-Zone

- Within single region, low latency
- Protects against rack/zone failures
- Lower cost, simpler implementation
- Vulnerable to region-wide outages

Multi-Region

- True disaster recovery (DR) capability
- Active-Active (traffic split) or Active-Passive (warm standby)
- Defined by RTO (recovery time) & RPO (data loss tolerance)
- Higher cost, data synchronization challenges

HA for GPU Clusters

GPU resources require special HA considerations due to their cost and workload characteristics:

- Distribute **GPU node pools** across AZs to avoid zone-based outages
- Implement job checkpointing (DDP/FSDP/DeepSpeed) to resume after node failures
- Use **elastic training frameworks** like PyTorch Elastic and Ray Train
- For inference, deploy multiple GPU pods with autoscaling rather than relying on single large GPUs



Modern ML frameworks support elastic execution
and failure recovery

Monitoring HA Health

Heartbeat Monitoring

Implement continuous health checks with Prometheus blackbox exporter to detect service availability issues

- Endpoint response time tracking
- SSL certificate expiration monitoring
- DNS resolution verification

Error Budget Tracking

Define and track Service Level Indicators (SLIs) and Objectives (SLOs)

- Response time percentiles (p50, p95, p99)
- Request success rate percentages
- Remaining error budget visualization

Alert Configuration

Set up proactive alerts for early warning signs

- Pod restart events exceeding thresholds
- Node NotReady conditions
- Load balancer health check failures
- Resource utilization approaching limits

HA Dashboards

Create dedicated dashboards showing availability metrics

- Overall uptime percentage
- Failover event history and duration
- Recovery time metrics
- Regional health comparison

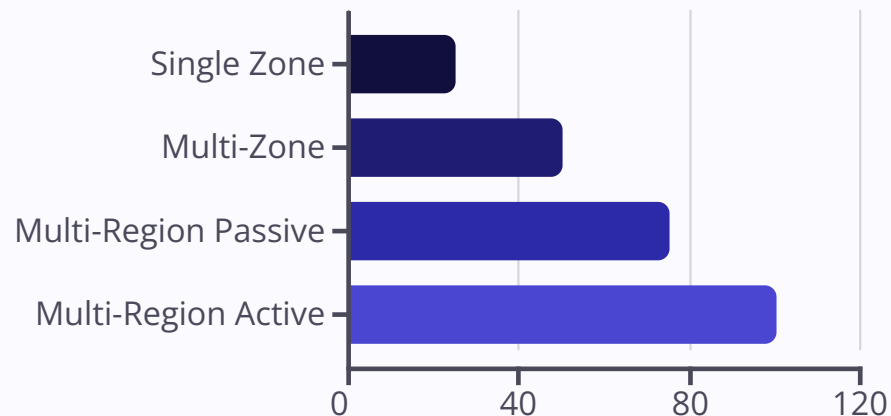
Cost Trade-offs for HA

The Cost of High Availability

HA comes with significant cost implications that must be balanced against business requirements:

- Multi-region storage replication can **double storage costs**
- Idle standby clusters represent wasted resources if rarely used
- Control plane redundancy increases management overhead
- Increased complexity requires more specialized skills

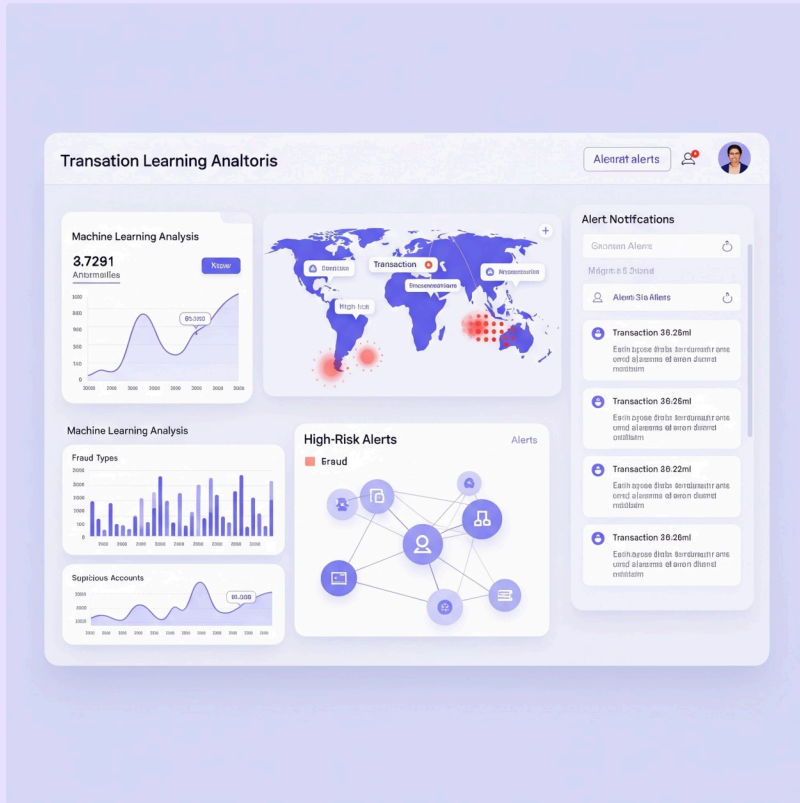
Balancing Cost vs SLA



Startup Capstone: Multi-zone only

Enterprise Capstone: Multi-region + DR drills

Case Study: Fraud Detection API



Critical Financial Services API

High-stakes real-time transaction analysis requiring consistent availability

HA Implementation

- Multi-zone Kubernetes cluster with 3 API replicas per service
- Database replication across 2 geographic regions
- Global load balancer with automated DNS failover
- Circuit breakers for dependent services

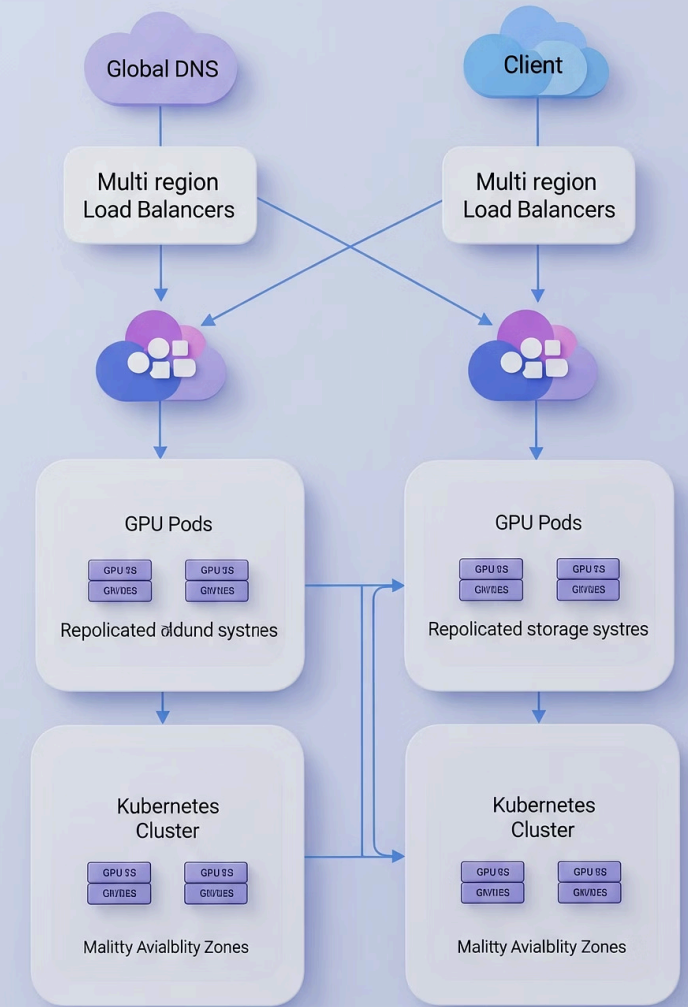
Results

Successfully survived a complete AZ outage with minimal disruption, maintaining **99.97% uptime** over the quarter

High Availability Architecture

HA = redundancy + failover across all infrastructure layers

This architecture eliminates single points of failure by distributing components geographically and implementing automated failover mechanisms at each layer. Client requests are routed through redundant global DNS to multi-region load balancers, which direct traffic to healthy Kubernetes clusters running in multiple availability zones.



Key Lessons for AI Infrastructure HA

01

HA requires comprehensive strategy

High availability is built on redundancy, failover, isolation, and self-healing mechanisms working together cohesively

02

All infrastructure layers must be covered

Compute, storage, networking, control plane, and application layers all require specific HA strategies

03

RTO/RPO requirements drive architectural decisions

Recovery Time Objective and Recovery Point Objective determine whether multi-zone or multi-region architectures are necessary

04

Capstone target: $\geq 99.9\%$ uptime with realistic costs

Balance reliability requirements with budget constraints to deliver a practical solution

Discussion: Capstone HA Strategy

Multi-Zone HA

Pros:

- Lower cost and complexity
- Easier data consistency
- Simpler networking
- Faster implementation timeline

Cons:

- Vulnerable to region-wide outages
- Limited geographic redundancy
- May not meet strict compliance requirements

Multi-Region HA

Pros:

- True disaster recovery capability
- Geographic redundancy for compliance
- Protection against region-wide failures
- Potential latency benefits for global users

Cons:

- Significantly higher cost
- Data synchronization challenges
- More complex networking and security
- Requires specialized expertise

For your Capstone, which approach would you choose and why?